MATERIA: INF560 — DESARROLLO WEB BACKEND

GUÍA DE LABORATORIO 3

Instalación del Entrono de Desarrollo

PHP – COMPOSER – LARAVEL – PostgreSQL

CONTENIDO DE LA GUÍA

1. Requisitos del entorno de desarrollo
2. Instalación de PHP 8.3+
3. Instalación de Composer
4. Instalación del instalador de Laravel
5. Instalación de PostgreSQL
6. Instalación de Node.js
7. Creación del primer proyecto Laravel 13
8. Configuración de PostgreSQL en Laravel
9. Verificación del entorno completo

COMPETENCIA DE LA GUÍA

Instalar y configurar un entorno de desarrollo completo para proyectos Laravel 13 con base de datos PostgreSQL, verificando la correcta integración de todos los componentes necesarios (PHP 8.3+, Composer, Node.js) en los sistemas operativos Windows, macOS y Linux.

1. Introducción

En esta guía de laboratorio instalaremos y configuraremos todas las herramientas necesarias para desarrollar aplicaciones web con Laravel 13 y PostgreSQL. Al finalizar, tendrás un entorno de desarrollo completamente funcional listo para los proyectos del curso.

¿QUÉ ES LARAVEL?
Laravel es un framework de código abierto para PHP que facilita el desarrollo de aplicaciones web robustas y escalables. Sigue el patrón de arquitectura MVC (Modelo-Vista-Controlador) y proporciona herramientas para enrutamiento, autenticación, migraciones de base de datos, colas de trabajo y mucho más.

¿POR QUÉ POSTGRESQL?
PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto, reconocido por su robustez, extensibilidad y cumplimiento del estándar SQL. Laravel 13 lo soporta nativamente y además ofrece funcionalidades avanzadas como búsqueda vectorial (pgvector) para proyectos con inteligencia artificial.

1.1. Componentes del entorno

El entorno de desarrollo para Laravel 13 se compone de los siguientes elementos:

Componente	Versión mínima	Función
PHP	8.3	Lenguaje de programación del servidor
Composer	2.x	Gestor de dependencias para PHP
Laravel Installer	Última	CLI para crear proyectos Laravel
Laravel Framework	13.x	Framework de desarrollo web
PostgreSQL	15+	Sistema de gestión de base de datos
Node.js	18+ (LTS)	Entorno para compilar assets frontend
NPM / Bun	Última	Gestor de paquetes JavaScript

IMPORTANTE
Laravel 13 requiere PHP 8.3 como versión mínima. Si tu sistema tiene una versión inferior (PHP 8.1 o 8.2), deberás actualizarla antes de continuar. Verifica tu versión actual con: php -v

2. Instalación de PHP 8.3+

PHP es el lenguaje de programación sobre el que se construye Laravel. Necesitamos la versión 8.3 o superior junto con varias extensiones requeridas por el framework.

2.1. Windows

En Windows existen dos opciones principales para instalar PHP: el método rápido con php.new o la instalación manual.

Opción A: Instalación rápida con php.new (recomendada)

Laravel ofrece un script que instala automáticamente PHP, Composer y el instalador de Laravel en un solo paso. Abre PowerShell como Administrador y ejecuta:

```
# Ejecutar en PowerShell como Administrador
Set-ExecutionPolicy Bypass -Scope Process -Force;
[System.Net.ServicePointManager]::SecurityProtocol =
  [System.Net.ServicePointManager]::SecurityProtocol -bor 3072;
iex ((New-Object System.Net.WebClient).DownloadString(
  'https://php.new/install/windows/8.4'
))
```

NOTA SOBRE PHP.NEW

El script php.new instala la versión más reciente de PHP (actualmente 8.5) que es compatible con Laravel 13. Después de ejecutarlo, reinicia tu terminal para que los cambios surtan efecto.

Opción B: Instalación manual

1 Descargar PHP
Visita https://windows.php.net/download/ y descarga la versión Thread Safe (x64) de PHP 8.3 o superior en formato ZIP.

2 Extraer archivos
Descomprime el ZIP en C:\php (o la carpeta que prefieras).

3 Configurar PATH
Agrega C:\php a la variable de entorno PATH del sistema: Panel de Control > Sistema > Configuración avanzada > Variables de entorno > Path > Nuevo.

4 Configurar php.ini
Copia php.ini-development a php.ini y habilita las extensiones necesarias descomentando (quitar el ;) las siguientes líneas:

```
; Abrir C:\php\php.ini y descomentar estas líneas:
extension=curl
extension=fileinfo
extension=mbstring
extension=openssl
extension=pdo_pgsql     ; Para PostgreSQL
extension=pgsql         ; Para PostgreSQL
extension=zip
extension=gd
extension=intl
extension=bcmath
extension=tokenizer
extension=xml
extension=ctype
```

5 **Verificar instalación**
Abre una nueva terminal (CMD o PowerShell) y ejecuta:

```
php -v
# Salida esperada:
# PHP 8.3.x (cli) ...
```

2.2. macOS

En macOS se recomienda usar Homebrew para instalar PHP. Si no tienes Homebrew, instálalo primero desde https://brew.sh.

Opción A: Con php.new (recomendada)

```
# Ejecutar en Terminal
/bin/bash -c "$(curl -fsSL https://php.new/install/mac/8.4)"
```

Opción B: Con Homebrew

```
# Instalar PHP 8.3
brew install php@8.3

# Agregar al PATH (si se instaló una versión específica)
echo 'export PATH="/opt/homebrew/opt/php@8.3/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

# Verificar
php -v
```

2.3. Linux (Ubuntu/Debian)

En distribuciones basadas en Debian/Ubuntu utilizaremos el repositorio PPA de Ondřej Surý para obtener versiones recientes de PHP.

Opción A: Con php.new (recomendada)

```
# Ejecutar en Terminal
/bin/bash -c "$(curl -fsSL https://php.new/install/linux/8.4)"
```

Opción B: Instalación manual con APT

```
# Agregar repositorio PPA
sudo add-apt-repository ppa:ondrej/php
sudo apt update

# Instalar PHP 8.3 con extensiones necesarias
sudo apt install php8.3 php8.3-cli php8.3-fpm \
  php8.3-pgsql php8.3-mbstring php8.3-xml \
  php8.3-curl php8.3-zip php8.3-bcmath \
  php8.3-intl php8.3-gd php8.3-tokenizer

# Verificar
php -v
```

EXTENSIONES PHP REQUERIDAS POR LARAVEL 13

Las extensiones imprescindibles son: ctype, curl, dom, fileinfo, filter, hash, mbstring, openssl, pcre, PDO, pdo_pgsql (para PostgreSQL), session, tokenizer y xml. La mayoría se instalan automáticamente con PHP; solo pdo_pgsql debe habilitarse manualmente.

3. Instalación de Composer

Composer es el gestor de dependencias estándar para PHP. Laravel y todos sus paquetes se distribuyen a través de Composer. Es indispensable para cualquier proyecto PHP moderno.

> **SI USASTE PHP.NEW**
> Si instalaste PHP mediante el script php.new (Opción A de la sección anterior), Composer ya está instalado. Puedes verificarlo con: composer --version. Si es así, salta directamente a la sección 4.

3.1. Windows (instalación manual)

1 **Descargar instalador**
Visita https://getcomposer.org/download/ y descarga Composer-Setup.exe.

2 **Ejecutar instalador**
Sigue el asistente. Asegúrate de que detecte correctamente la ruta de php.exe (C:\php\php.exe).

3 **Verificar**
Abre una nueva terminal y ejecuta:

```
composer --version
# Salida esperada:
# Composer version 2.x.x ...
```

3.2. macOS / Linux (instalación manual)

```
# Descargar e instalar Composer globalmente
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer
sudo chmod +x /usr/local/bin/composer

# Verificar
composer --version
```

4. Instalación del Instalador de Laravel

El instalador de Laravel es una herramienta CLI que permite crear nuevos proyectos Laravel de forma rápida y con opciones interactivas para elegir base de datos, framework de testing y starter kit.

> **SI USASTE PHP.NEW**
> El script php.new ya incluye el instalador de Laravel. Verifica con: laravel --version. Si el comando funciona, pasa a la sección 5.

4.1. Instalación vía Composer

Si no usaste php.new, instala el instalador globalmente con Composer:

```
composer global require laravel/installer
```

4.2. Configurar PATH del binario global de Composer

Para que el comando laravel esté disponible en tu terminal, asegúrate de que la carpeta de binarios globales de Composer esté en tu PATH:

```
# Windows: agregar al PATH del sistema:
# %USERPROFILE%\AppData\Roaming\Composer\vendor\bin

# macOS / Linux: agregar a ~/.zshrc o ~/.bashrc:
export PATH="$HOME/.config/composer/vendor/bin:$PATH"
# o (en algunos sistemas):
export PATH="$HOME/.composer/vendor/bin:$PATH"
```

4.3. Verificar

```
laravel --version
# Salida esperada:
# Laravel Installer X.X.X
```

5. Instalación de PostgreSQL

PostgreSQL será nuestro sistema de base de datos durante todo el curso. Instalaremos el servidor y la herramienta de administración pgAdmin para gestionar las bases de datos de forma visual.

5.1. Windows

1 Descargar instalador
Visita https://www.postgresql.org/download/windows/ y descarga el instalador de EnterpriseDB (EDB) para la versión 16 o superior.

2 Ejecutar instalador
Sigue el asistente de instalación. Cuando te pida, establece una contraseña para el superusuario postgres (anótala, la necesitarás después). Deja el puerto por defecto: 5432.

3 Incluir pgAdmin
El instalador de EDB incluye pgAdmin 4 automáticamente. Marca la casilla correspondiente si aparece.

4 Agregar al PATH
El instalador normalmente agrega PostgreSQL al PATH. Si no lo hace, agrega manualmente: C:\Program Files\PostgreSQL\16\bin

5 Verificar
Abre una nueva terminal y ejecuta:

```
psql --version
# Salida esperada:
# psql (PostgreSQL) 16.x
```

5.2. macOS

Opción A: Con Homebrew (recomendada)

```
# Instalar PostgreSQL
brew install postgresql@16

# Iniciar el servicio
brew services start postgresql@16

# Verificar
psql --version
```

Opción B: Con Postgres.app

Descarga Postgres.app desde https://postgresapp.com. Es una aplicación nativa de macOS que incluye PostgreSQL preconfigurado. Solo descarga, arrastra a Aplicaciones y ejecuta.

5.3. Linux (Ubuntu/Debian)

```
# Instalar PostgreSQL
sudo apt update
sudo apt install postgresql postgresql-contrib

# El servicio se inicia automáticamente
# Verificar estado
sudo systemctl status postgresql

# Verificar versión
psql --version
```

5.4. Configuración inicial de PostgreSQL

Después de instalar PostgreSQL, debemos crear un usuario y una base de datos para nuestros proyectos Laravel.

Windows (desde cmd o PowerShell)

```
# Conectarse como superusuario
psql -U postgres
# (ingresa la contraseña que definiste en la instalación)

-- Crear un usuario para desarrollo
CREATE USER laravel_user WITH PASSWORD 'secreto123';

-- Crear una base de datos para el curso
CREATE DATABASE inf560_db OWNER laravel_user;

-- Otorgar todos los privilegios
GRANT ALL PRIVILEGES ON DATABASE inf560_db TO laravel_user;

-- Salir
\q
```

macOS / Linux

```
# En Linux, primero cambiar al usuario postgres
sudo -u postgres psql

# En macOS con Homebrew, directamente:
psql postgres

-- Crear usuario y base de datos
CREATE USER laravel_user WITH PASSWORD 'secreto123';
CREATE DATABASE inf560_db OWNER laravel_user;
GRANT ALL PRIVILEGES ON DATABASE inf560_db TO laravel_user;

-- Salir
\q
```

SEGURIDAD DE CONTRASEÑAS

La contraseña 'secreto123' es solo un ejemplo para desarrollo local. En un entorno de producción NUNCA uses contraseñas simples. Para el curso, puedes usar esta contraseña de ejemplo o definir una propia; lo importante es recordarla para configurarla en Laravel.

VERIFICAR LA CONEXIÓN

Para verificar que el usuario y la base de datos se crearon correctamente, intenta conectarte con el nuevo usuario:

```
psql -U laravel_user -d inf560_db -h localhost
```

Si la conexión es exitosa verás el prompt: `inf560_db=>`

6. Instalación de Node.js

Node.js es necesario para compilar los assets del frontend (CSS, JavaScript) mediante Vite, que es la herramienta de compilación que usa Laravel por defecto. Se necesita Node.js 18 o superior (versión LTS recomendada).

6.1. Windows

1 **Descargar instalador**
Visita https://nodejs.org y descarga la versión LTS (Long Term Support). Al momento de esta guía, la versión LTS es 22.x.

2 **Ejecutar instalador**
Sigue el asistente. Acepta los valores predeterminados. El instalador agrega Node.js y NPM al PATH automáticamente.

3 **Verificar**
Abre una nueva terminal:

```
node -v
# Salida esperada: v22.x.x

npm -v
# Salida esperada: 10.x.x
```

6.2. macOS

```
# Con Homebrew
brew install node

# Verificar
node -v
npm -v
```

6.3. Linux (Ubuntu/Debian)

```
# Usando NodeSource (versión LTS)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs

# Verificar
node -v
npm -v
```

ALTERNATIVA: BUN

> Laravel también soporta Bun como alternativa a Node.js y NPM. Bun es un runtime de JavaScript más rápido. Si deseas usarlo, instálalo desde https://bun.sh. Sin embargo, para el curso utilizaremos Node.js y NPM por ser más estándar y ampliamente documentado.

7. Creación del Primer Proyecto Laravel 13

Con todas las herramientas instaladas, es momento de crear nuestro primer proyecto Laravel 13 y verificar que todo funcione correctamente.

7.1. Crear el proyecto

Abre tu terminal y navega a la carpeta donde deseas crear el proyecto. Luego ejecuta:

```
# Método 1: Con el instalador de Laravel (recomendado)
laravel new mi-proyecto

# Método 2: Con Composer directamente
composer create-project laravel/laravel:^13.0 mi-proyecto
```

OPCIONES INTERACTIVAS DEL INSTALADOR

Al usar laravel new, el instalador te hará varias preguntas interactivas:

Testing framework: Selecciona Pest o PHPUnit (para el curso usaremos PHPUnit).

Database: Selecciona PostgreSQL.

Starter kit: Selecciona None si quieres un proyecto limpio, o escoge un starter kit según la necesidad.

7.2. Verificar la versión de Laravel

```
cd mi-proyecto
php artisan --version
# Salida esperada:
# Laravel Framework 13.x.x
```

7.3. Estructura del proyecto

Una vez creado, el proyecto tendrá la siguiente estructura de carpetas principales:

Carpeta / Archivo	Descripción
app/	Código de la aplicación (modelos, controladores, servicios)
bootstrap/	Archivos de inicialización del framework
config/	Archivos de configuración (database, mail, cache, etc.)
database/	Migraciones, factories y seeders
public/	Punto de entrada web (index.php) y assets públicos
resources/	Vistas Blade, archivos CSS/JS sin compilar

routes/	Definición de rutas (web.php, api.php, console.php)
storage/	Archivos generados (logs, cache, sesiones, uploads)
tests/	Tests unitarios y de integración
.env	Variables de entorno (configuración local)
composer.json	Dependencias PHP del proyecto
package.json	Dependencias JavaScript del proyecto

8. Configuración de PostgreSQL en Laravel

Ahora conectaremos nuestro proyecto Laravel con la base de datos PostgreSQL que creamos en la sección 5. La configuración se realiza en el archivo .env ubicado en la raíz del proyecto.

8.1. Editar el archivo .env

Abre el archivo .env con tu editor de texto favorito (VS Code, Sublime Text, etc.) y modifica las siguientes variables de base de datos:

```
# ─── Configuración de Base de Datos ───
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=inf560_db
DB_USERNAME=laravel_user
DB_PASSWORD=secreto123
```

ARCHIVO .ENV Y CONTROL DE VERSIONES

El archivo .env contiene información sensible como contraseñas y claves de API. NUNCA debes subir este archivo a repositorios Git. Laravel ya incluye .env en el archivo .gitignore por defecto. Cada desarrollador debe tener su propio archivo .env local.

8.2. Verificar la conexión

Para verificar que Laravel puede conectarse a PostgreSQL, ejecuta las migraciones iniciales:

```
php artisan migrate

# Salida esperada:
# INFO  Running migrations.
#
#   0001_01_01_000000_create_users_table ......... DONE
#   0001_01_01_000001_create_cache_table ......... DONE
#   0001_01_01_000002_create_jobs_table .......... DONE
```

¿QUÉ SON LAS MIGRACIONES?

Las migraciones son archivos PHP que definen la estructura de las tablas de la base de datos. Laravel incluye migraciones por defecto para usuarios, cache y colas de trabajo. El comando php artisan migrate ejecuta estas migraciones creando las tablas correspondientes en PostgreSQL.

8.3. Verificar en pgAdmin

Opcionalmente, puedes abrir pgAdmin 4 para verificar visualmente que las tablas se crearon:

1

Abrir pgAdmin 4
Busca pgAdmin en tu menú de aplicaciones y ábrelo.

2

Conectar al servidor
En el panel izquierdo, expande Servers > PostgreSQL > Databases > inf560_db.

3

Verificar tablas
Navega a inf560_db > Schemas > public > Tables. Deberías ver las tablas: users, password_reset_tokens, sessions, cache, cache_locks, jobs, job_batches, failed_jobs.

9. Verificación del Entorno Completo

Antes de dar por finalizada la instalación, realicemos una verificación completa del entorno y probemos que el servidor de desarrollo funciona correctamente.

9.1. Checklist de verificación

Ejecuta cada uno de los siguientes comandos y verifica que obtienes la salida esperada:

```
# 1. PHP
php --version          # Debe mostrar PHP 8.3.x o superior

# 2. Composer
composer --version     # Debe mostrar Composer version 2.x.x

# 3. Laravel Installer
laravel --version      # Debe mostrar Laravel Installer X.X.X

# 4. Node.js y NPM
node --version         # Debe mostrar v18.x.x o superior
npm --version          # Debe mostrar 9.x.x o superior

# 5. PostgreSQL
psql --version         # Debe mostrar psql (PostgreSQL) 15.x o superior

# 6. Laravel (dentro del proyecto)
cd mi-proyecto
php artisan --version  # Debe mostrar Laravel Framework 13.x.x
```

9.2. Iniciar el servidor de desarrollo

Laravel incluye un comando que inicia simultáneamente el servidor PHP, el worker de colas y el servidor Vite para compilación de assets:

```
# Instalar dependencias JavaScript
npm install

# Iniciar servidor de desarrollo
composer run dev

# O de forma individual:
php artisan serve     # Servidor PHP en http://localhost:8000
```

SERVIDOR DE DESARROLLO

El comando composer run dev ejecuta tres procesos en paralelo: el servidor HTTP de Laravel (artisan serve), el worker de colas (artisan queue:work) y el servidor Vite (npm run dev). Accede a http://localhost:8000 en tu navegador para ver la página de bienvenida de Laravel.

9.3. Probar la conexión desde el navegador

Abre tu navegador y accede a http://localhost:8000. Deberías ver la página de bienvenida de Laravel 13. Si ves esta página, tu entorno está completamente configurado y funcionando.

9.4. Editor de código recomendado

Para trabajar con Laravel se recomienda Visual Studio Code con las siguientes extensiones:

Extensión	Función
PHP Intelephense	Autocompletado y análisis de código PHP
Laravel Blade Snippets	Resaltado de sintaxis y snippets para Blade
Laravel Extra Intellisense	Autocompletado para rutas, vistas, config
DotENV	Resaltado de sintaxis para archivos .env
PostgreSQL (de Chris Kolkman)	Explorador y consultas SQL para PostgreSQL
Thunder Client o REST Client	Probar endpoints API directamente desde VS Code

10. Solución de Problemas Comunes

A continuación se listan los errores más frecuentes durante la instalación del entorno y sus soluciones.

Error	Causa	Solución
php: command not found	PHP no está en el PATH del sistema	Agrega la carpeta de PHP al PATH y reinicia la terminal
composer: command not found	Composer no está instalado o no está en el PATH	Reinstala Composer o agrega su ruta al PATH
laravel: command not found	El binario global de Composer no está en el PATH	Agrega ~/.config/composer/vendor/bin al PATH
could not find driver (PDOException)	La extensión pdo_pgsql no está habilitada	Habilita pdo_pgsql en php.ini o instala el paquete php8.3-pgsql
SQLSTATE[08006] Connection refused	PostgreSQL no está ejecutándose o el puerto es incorrecto	Verifica que el servicio PostgreSQL esté activo y usa el puerto 5432
password authentication failed	Contraseña incorrecta en .env	Verifica DB_PASSWORD en .env contra la contraseña del usuario PostgreSQL
database "X" does not exist	La base de datos no fue creada	Crea la base de datos con CREATE DATABASE desde psql
npm: command not found	Node.js no está instalado	Instala Node.js desde nodejs.org
Your requirements could not be resolved	Versión de PHP incompatible	Actualiza PHP a la versión 8.3 o superior

11. Bibliografía y Recursos

Documentación oficial de Laravel 13. Disponible en: https://laravel.com/docs/13.x
Guía de instalación de Laravel 13. Disponible en: https://laravel.com/docs/13.x/installation
Documentación oficial de PostgreSQL. Disponible en: https://www.postgresql.org/docs/
Documentación de Composer. Disponible en: https://getcomposer.org/doc/
Documentación de Node.js. Disponible en: https://nodejs.org/docs/latest/api/
PHP.net — Manual de PHP. Disponible en: https://www.php.net/manual/es/
Laravel News — Laravel 13 Release. Disponible en: https://laravel-news.com/laravel-13